Lctr 20: Immediate-Offset Load and Store Instructions and Macro

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Immediate-offset load and store instructions for different data types
  - 32-bit signed and unsigned
  - 16-bit signed and unsigned
  - 8-bit signed and unsigned
- C pointer operations implemented by the above instructions.

Demo project:

- Illustration of example problems of load and store.

C and ASM programming:

- Macros in ASM for clean coding. (Can be skipped.)

20.1 Introduction

Load and store instructions are used for accessing the data stored in memory.

There are **two main aspects** for this type of data access using load and store instructions:

- **The value of the pointer, used as the address**. We use a (general-purpose) register **Ra** to signify the **address** in the load and store instructions in the sequel.
- **The value of the variable, the dereference of the pointer**. We use a register **Rv** to signify the **value** in the load and store instructions in the sequel.

20.1.1 The demo project

The load and store instructions are closely related to C's pointer operations. These instructions and concepts are very important but can be a bit difficult to understand. As such, we will use many small functions in the demo project, `gc1s_ldr_n_str_n_macro`. The ASM functions are collected in the `gc1s_ldr_n_str_n_macro_sfns.s` file of the demo project. For better readability, we have provided moderate comments for the important lines.

20.2 Immediate-offset load instructions

When dereferencing a C pointer, say `p32`, we can add more operations to the pointer, such as

- A constant offset: such as `*(p32+2)`.
- Prefix increment or decrement: `*++p32` or `*--p32`.
- Postfix increment or decrement: `*p32++` or `*p32--`.

20.2.1 Three types of immediate-offset load instructions

For ASM, we have:

- The basic immediate-offset LDR (LoaD Register) instruction:
  - Syntax: `LDR Rv, [Ra{, #os}]`. Note that the register used for addressing is enclosed in the brackets!
  - A single-step operation: load the word **pointed** by Ra + `os` to register Rv.

  - Equivalent C operation: `var = *(p32+ k)` for `os = 4*k`. Assume R0 = `var` and R1 = `p32`:
    * `var = *(p32+ 2)` <==> `ldr r0, [r1, #8]`.

- Pre-indexed immediate-offset load instruction:
  - Syntax: `LDR Rv, [Ra, #os]!`.
  - A sequential two-step operation (without using additional cycles):
    * Step 1. Increment the value of the address register: `ADD Ra, #os`.
    * Step 2. Load the value as if running: `LDR Rv, [Ra]`.
  - Also assume R0 = `var` and R1 = `p32`:
    * `var = *++p32` <==> `ldr r0, [r1, #4]!`.
    * `var = *--p32` <==> `ldr r0, [r1, #-4]!`.

- Post-indexed immediate-offset load instruction:
  - Syntax: `LDR Rv, [Ra], #os`.
  - A sequential two-step operation (without using additional cycles):
    * Step 1. Load the value as if running: `LDR Rv, [Ra]`.
    * Step 2. Increment the value of the address register: `ADD Ra, #os`.
  - Again assume assume R0 = `var` and R1 = `p32`:
    * `var = *p32++` <==> `ldr r0, [r1], #4`.
    * `var = *p32--` <==> `ldr r0, [r1], #-4`.

Note: Variable `var` can be both signed and unsigned.

How to memorize the different notations:

- `ldr 0, [r1, #4]`: Without anything after the closing bracket, we do nothing on the address register. So its value is not updated.
- `ldr 0, [r1, #4]!`: With an **exclamation mark** after the closing bracket, we need to update the address register **urgently**! So, **we update the address register before using it**.
- `ldr 0, [r1], #4`: With a number for offset after (trailing) the closing bracket, we can take our time to update it, so we update it after the main business, the loading of the data.

20.2.2 Examples and demo code

	Byte 0	Byte 1
0x2000_0200	0x70	0x72
0x2000_0202	0x74	0x76
0x2000_0204	0x78	0x7A
0x2000_0206	0x7C	0x7E
0x2000_0208	0x80	0x82
0x2000_020A	0x84	0x86
0x2000_020C	0x88	0x8A
0x2000_020E	0x8C	0x8E

Figure 1: Memory map.

Example 1. Given the memory map shown in Figure 1. Write the C code using a pointer to assign the above values, assuming the memory block is well reserved. Also translate the C code to ASM with starting address changed to `0x2000_0210`.

Solution.

The C function used to initialize values is shown below:

```c
void mp_init_mem_block(void) {
    uint8_t *p = (uint8_t *)0x20000200;
    for (int i = 0; i < 16; i++) {
        *p++ = 0x70 + (i << 1);
    }
}
```

The ASM translation (with changed address) is shown below:

```
@ void mp_init_mem_block_s(void) {
@     uint8_t *p = (uint8_t *)0x20000210;
@     for (int i = 0; i < 16; i++) {
@         *p++ = 0x70 + (i << 1);
@     }
@ }
mp_init_mem_block_s:
    ldr     r0, =0x20000210     @ *p = (uint8_t *)0x20000210
    ldr     r1, =0              @ int i = 0
    ldr     r2, =0x70           @ temp1 = 0x70
mp_init_mem_block_s_loop:
    cmp     r1, #16
    bge     mp_init_mem_block_s_end
    add     r3, r2, r1, lsl #1  @ temp2 = temp1 + (i << 1)
    strb    r3, [r0], #1        @ *p++ = temp2
    add     r1, #1              @ i++
    b       mp_init_mem_block_s_loop
mp_init_mem_block_s_end:
    bx      lr
```

> -:-

	Byte 0	Byte 1
0x2000_0200	0x70	0x72
0x2000_0202	0x74	0x76
0x2000_0204	0x78	0x7A
0x2000_0206	0x7C	0x7E
0x2000_0208	0x80	0x82
0x2000_020A	0x84	0x86
0x2000_020C	0x88	0x8A
0x2000_020E	0x8C	0x8E

Figure 2: Memory map.

Example 2. (Immediate-offset loading of 32-bit numbers.) Given the memory map shown in Figure 1 and the values of three registers for addresses:

- R1 = 0x2000_0200
- R2 = 0x2000_0204
- R3 = 0x2000_020C

Determine the value of R1 to R6 after running the following code

```
    ldr     r4, [r1, #8]
    ldr     r5, [r2, #-4]!
    ldr     r6, [r3], #12
```

Solution:

Note that the above results can be verified by the following code, duplicated from `gc1s_ldr_n_str_n_macro_sfns.s` of the demo project. For better readability, we have provided moderate comments for the important lines:

```
@ void mp_32_bit_immediate_offset_load(int *Ra)
@   Input:
@     * Ra -> r0 (Address)
@   Output:
@     * None
mp_32_bit_immediate_offset_load:
    push    {r4-r6, lr}         @ Push related Regs in stack
    initialize_r1_to_r6_local   @ Init for stressing changes

    @ Set values to the address registers
    ldr     r1, =0x20000200
    ldr     r2, =0x20000204
    ldr     r3, =0x2000020C

    store_r1_to_r6_local 0  @ Store registers before operations

    @ Read via using LDR    @ Load instruction of the problem
    ldr     r4, [r1, #8]
    ldr     r5, [r2, #-4]!
    ldr     r6, [r3], #12

    store_r1_to_r6_local 24 @ Store registers after operations
    pop     {r4-r6, pc}     @ Restore values of Regs and return
```

20.3 Variants in data type of the basic immediate-offset load instructions

20.3.1 Extending immediate-offset load instructions to 8- and 16-bit numbers

The above basic load instructions are used for loading 32-bit signed or unsigned numbers from the memory. What if the data are stored in width of 8 or 16 bits?

Each of the above load instructions can be attached to a data *type*:

```
    LDR{type} Rv, [Ra]
```

where {type} can be one of:

- `B`, for **unsigned byte**, which needs a **zero** extension to higher bits. So, the value of a byte 0xFF will be extended to a word 0x0000_00FF; both are 255.
- `SB`, for **signed byte**, which needs a **sign** extension to higher bits. So, the value of a byte 0xFF will be extended to a word 0xFFFF_FFFF; both are -1. On the other hand, the value of a byte 0x7F will be extended to a word 0x0000_007F; both are 127.
- `H`, for **unsigned halfword**, which needs a **zero** extension to higher bits. So, the value of a byte 0xFFFF will be extended to a word 0x0000_FFFF; both are 65535.
- `SH`, for **signed halfword**, which needs a **sign** extension to higher bits. So, the value of a byte 0xFFFF will be extended to a word 0xFFFF_FFFF; both are -1. On the other hand, the value of a byte 0x7FFF will be extended to a word 0x0000_7FFF; both are 32767.
- − (omitted), for word; no change is introduced.

	Byte 0	Byte 1
0x2000_0200	0x70	0x72
0x2000_0202	0x74	0x76
0x2000_0204	0x78	0x7A
0x2000_0206	0x7C	0x7E
0x2000_0208	0x80	0x82
0x2000_020A	0x84	0x86
0x2000_020C	0x88	0x8A
0x2000_020E	0x8C	0x8E

Figure 3: Memory map.

Example 3. (Immediate-offset loading of 8- and 16-bit signed numbers.) Given the memory map shown in Figure 1 and the values of three registers for addresses:

- R1 = 0x2000_0204
- R2 = 0x2000_0206
- R3 = 0x2000_0208

Determine the value of R1 to R6 after running the following code

```
    ldrsb     r4, [r1, #10]
    ldrsh     r5, [r2, #-2]!
    ldrsh     r6, [r3], #8
```

Solution.

20.3.2 Mapping operations of C pointers for 8- and 16-bit numbers and their ASM counterparts

We have shown the mapping between operations of C pointers for 32-bit numbers and their ASM counterparts earlier. Now, let's consider the mapping of similar operations for 8- and 16-bit numbers and their ASM counterparts. Assume we have the following four pointers for different data types:

- `uint8_t *pu8`
- `int8_t *pi8`
- `uint16_t *pu16`
- `int16_t *pi16`

Similar to the above, we assume R0 is used for holding the data and R1 the address.

Now, we have the following:

- `var = *(pu8+ k)` (where `k` is a constant) is equivalent to `ldrb r0, [r1+ #os]` for `os = k`. Note that `os` is the same as `k` since each element of the array pointed by `pu8` is only ONE byte.
- `var = *(pi8+ k)` is equivalent to `ldrsb r0, [r1+ #os]` for `os = k`.
- `var = *(pu16+ k)` is equivalent to `ldrh r0, [r1+ #os]` for `os = 2*k`. Here `os = 2*k` since each element of the array pointed by `pu16` is a halfword, which has two bytes.
- `var = *(pi16+ k)` is equivalent to `ldrsh r0, [r1+ #os]` for `os = 2*k`.

A summary of C and ASM equivalences:

C pointer operation	Load instruction	Note
`var = *(pu8+ k)`	`ldrb r0, [r1, #os]`	`os = k`
`var = *(pi8+ k)`	`ldrsb r0, [r1, #os]`	`os = k`
`var = *++pu8`	`ldrb r0, [r1, #1]!`	
`var = *--pu8`	`ldrb r0, [r1, #-1]!`	
`var = *pu8++`	`ldrb r0, [r1], #1`	
`var = *pu8--`	`ldrb r0, [r1], #-1`	
`var = *++pi8`	`ldrsb r0, [r1, #1]!`	
`var = *--pi8`	`ldrsb r0, [r1, #-1]!`	
`var = *pi8++`	`ldrsb r0, [r1], #1`	
`var = *pi8--`	`ldrsb r0, [r1], #-1`	
`var = *(pu16+ k)`	`ldrh r0, [r1, #os]`	`os = 2*k`
`var = *(pi16+ k)`	`ldrsh r0, [r1, #os]`	`os = 2*k`
`var = *++pu16`	`ldrh r0, [r1, #2]!`	
`var = *--pu16`	`ldrh r0, [r1, #-2]!`	
`var = *pu16++`	`ldrh r0, [r1], #2`	
`var = *pu16--`	`ldrh r0, [r1], #-2`	
`var = *++pi16`	`ldrsh r0, [r1, #2]!`	
`var = *--pi16`	`ldrsh r0, [r1, #-2]!`	
`var = *pi16++`	`ldrsh r0, [r1], #2`	
`var = *pi16--`	`ldrsh r0, [r1], #-2`	

20.4 Immediate-offset store instructions

Store instructions are just the opposite of the load instructions, with only one exception: when we store the value as an 8- or 16-bit number, we don't have two distinct forms for unsigned and signed. We only have the unsigned form. The reason is that the processor assumes that the programmer has prepared the data appropriately at the lowest 8 or 16 bits of the register.

So, we have the following instructions, with `type` being `B` for byte storing and `H` for halfword:

- The basic immediate-offset store instruction:
  - Syntax: `STR{type} Rv, [Ra{, #os}]`. (If offset `os` is zero, we can leave it out.)
  - A single-step operation: store the 32-, 16-, or 8-bit number held in Rv to the memory with the address pointed by Ra + `os`.
- Pre-indexed immediate-offset store instruction:
  - Syntax: `STR{type} Rv, [Ra, #os]!`. (Here offset `os` cannot be zero as we need to update Ra using it.)
  - A sequential two-step operation (without using additional cycles):
    - Step 1. Increment the value of the address register as if running: `ADD Ra, #os`.
    - Step 2. Store the value as if running: `STR{type} Rv, [Ra]`.
- Post-indexed immediate-offset store instruction:
  - Syntax: `STR{type} Rv, [Ra], #os`.
  - A sequential two-step operation (without using additional cycles):
    - Step 1. Store the value as if running: `LDR{type} Rv, [Ra]`.
    - Step 2. Increment the value of the address register as if running: `ADD Ra, #os`.

Now, let's take a look at the equivalence between the C pointer operations and the store instructions. Note that we do not distinguish the signed and unsigned numbers when storing. So, we use only three types of pointers, which can be for both signed and unsigned numbers:

- For 32-bit numbers: `p32`
- For 16-bit numbers: `p16`
- For 8-bit numbers: `p8`

C pointer operation	Store instruction	Note
`*(p32+k)= var`	`str r0, [r1, #os]`	`os = 4*k`
`*++p32= var`	`str r0, [r1, #4]!`	
`*--p32= var`	`str r0, [r1, #-4]!`	
`*p32++ = var`	`str r0, [r1], #4`	
`*p32-- = var`	`str r0, [r1], #-4`	
`*(p16+k)= var`	`strh r0, [r1, #os]`	`os = 2*k`
`*++p16= var`	`strh r0, [r1, #2]!`	
`*--p16= var`	`strh r0, [r1, #-2]!`	
`*p16++ = var`	`strh r0, [r1], #2`	
`*p16-- = var`	`strh r0, [r1], #-2`	
`*(p8+k)= var`	`strb r0, [r1, #os]`	`os = k`
`*++p8= var`	`strb r0, [r1, #1]!`	
`*--p8= var`	`strb r0, [r1, #-1]!`	
`*p8++ = var`	`strb r0, [r1], #1`	
`*p8-- = var`	`strb r0, [r1], #-1`	

20.5 Contrasting data types in load and store instructions

We have mentioned that store instructions are somewhat simpler than the corresponding load instructions as we have less cases to consider. If you understand the related topics already, you can skip this part. If a bit help is needed, here we contract them with further explanations.

- For 32-bit numbers:
  - To load a 32-bit number, we use `ldr r0, [r1]`.
  - To store a -bit number, we use `str r0, [r1]`.
- For 16-bit numbers:
  - To load an `uint16_t` number, we use `ldrh r0, [r1]`.
  - To load an `int16_t` number, we use `ldrsh r0, [r1]`.
  - To store a 16-bit number, we use `strh r0, [r1]`.
- For 8-bit numbers:
  - To load an `uint8_t` number, we use `ldrb r0, [r1]`.
  - To load an `int8_t` number, we use `ldrsb r0, [r1]`.
  - To store an 8-bit number, we use `strb r0, [r1]`.